FIFO (Named Pipes)

```
achz25@DESKTOP-L8GIS7I:~$ mkfifo ilmiotubo
achz25@DESKTOP-L8GIS7I:~$ stat ilmiotubo
  File: ilmiotubo
  Size: 0               Blocks: 0          IO Block: 4096   fifo
Device: 8,32    Inode: 2386        Links: 1
Access: (0644/prw-r--r--)  Uid: ( 1000/  achz25)   Gid: ( 1000/  achz25)
Access: 2025-11-04 02:34:50.654848412 +0100
Modify: 2025-11-04 02:34:50.654848412 +0100
Change: 2025-11-04 02:34:50.654848412 +0100
 Birth: 2025-11-04 02:34:50.654848412 +0100
achz25@DESKTOP-L8GIS7I:~$
```

FIFO (Named Pipes)

```
achz25@DESKTOP-L8GIS7I:~$ mkfifo ilmiotubo
achz25@DESKTOP-L8GIS7I:~$ stat ilmiotubo
  File: ilmiotubo
  Size: 0               Blocks: 0          IO Block: 4096   fifo
Device: 8,32    Inode: 2386        Links: 1
Access: (0644/prw-r--r--)  Uid: ( 1000/  achz25)   Gid: ( 1000/  achz25)
Access: 2025-11-04 02:34:50.654848412 +0100
Modify: 2025-11-04 02:34:50.654848412 +0100
Change: 2025-11-04 02:34:50.654848412 +0100
 Birth: 2025-11-04 02:34:50.654848412 +0100
achz25@DESKTOP-L8GIS7I:~$
```

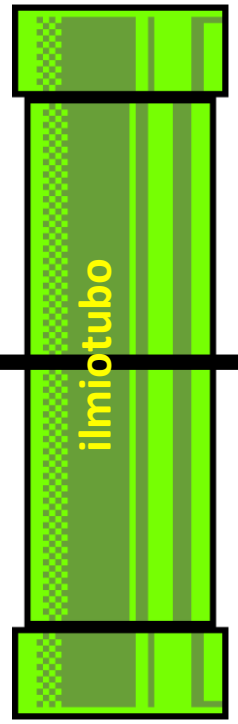
Una FIFO (First In, First Out) è un tipo speciale di file che serve per comunicare tra processi diversi, cioè per la comunicazione interprocesso (IPC).

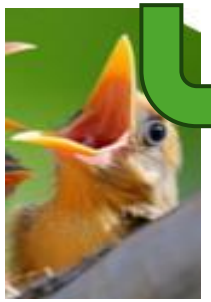
È chiamata "pipe nominata" perché, a differenza delle pipe anonime (|), ha un nome nel filesystem.

Funziona come una coda: il primo dato scritto è il primo che viene letto (da qui il nome FIFO).

```
$ echo "Ciao dal terminale A!" > ./ilmiotubo
```

Terminale A (mittente)





Terminale B (ricevente)

```
$ cat ilmiotubo
$ cat < ilmiotubo
Ciao dal terminale A!
```

Esempio bidirezionale (semplice chat)

Creare due pipe per una comunicazione a due vie:

Bash

```bash
mkfifo pipe_AtoB pipe_BtoA
```

Bash - TERMINALE A

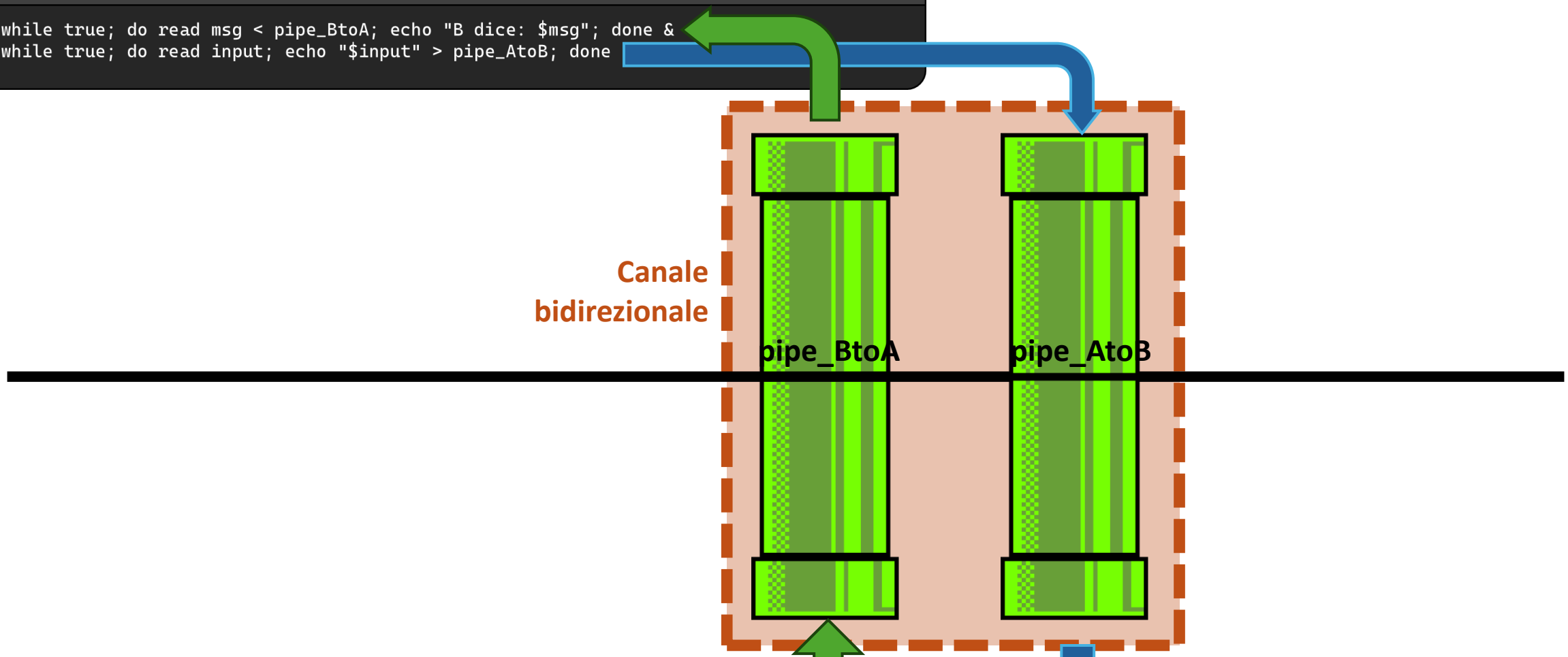
```bash
while true; do read msg < pipe_BtoA; echo "B dice: $msg"; done &
while true; do read input; echo "$input" > pipe_AtoB; done
```

Bash - TERMINALE B

```bash
while true; do read msg < pipe_AtoB; echo "A dice: $msg"; done &
while true; do read input; echo "$input" > pipe_BtoA; done
```

Bash - TERMINALE A

```
while true; do read msg < pipe_BtoA; echo "B dice: $msg"; done &
while true; do read input; echo "$input" > pipe_AtoB; done
```



Bash - TERMINALE B

```
while true; do read msg < pipe_AtoB; echo "A dice: $msg"; done &
while true; do read input; echo "$input" > pipe_BtoA; done
```

Comunicazione con pipe e netcat

Bash

```
mkfifo pipe_nc
```

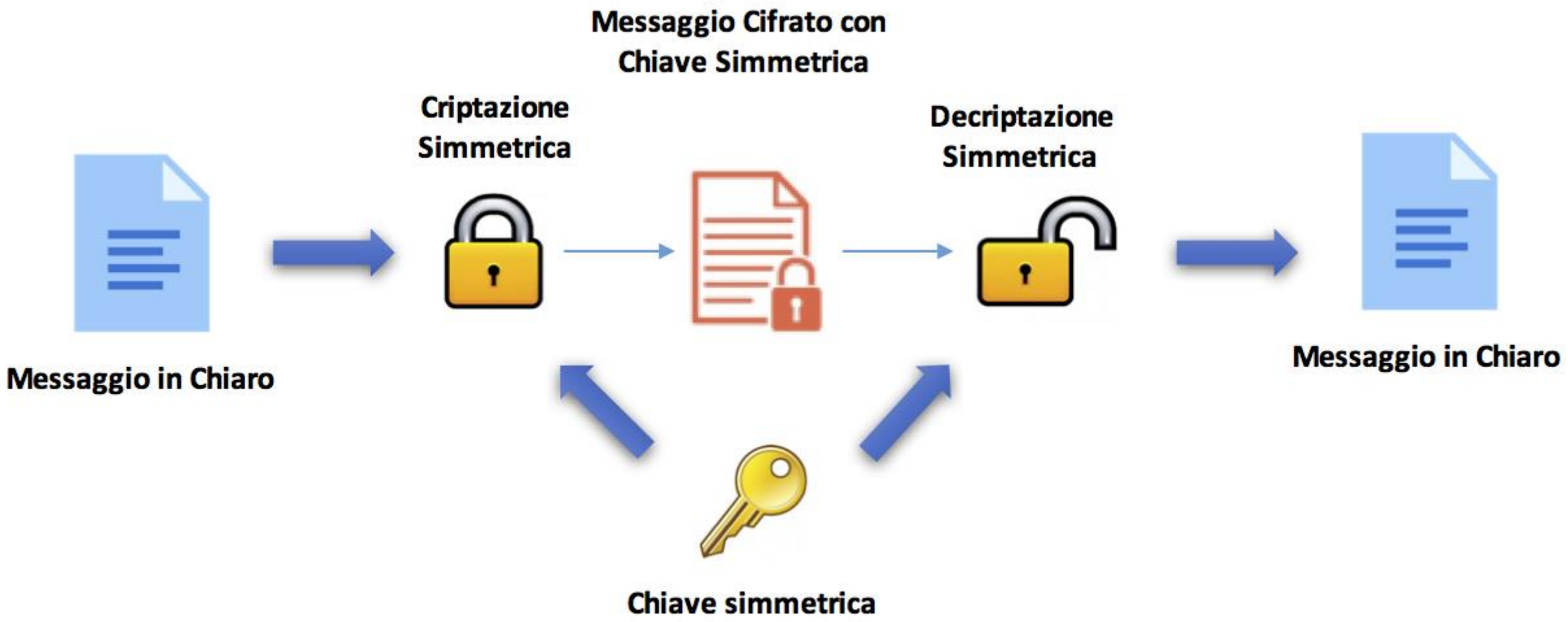
Bash - TERMINALE Server, Avvia il server nc, collegato alla pipe e invia dati al client scrivendo alla pipe con cat:

```
nc -l 1234 < pipe_nc &
cat > pipe_nc
```

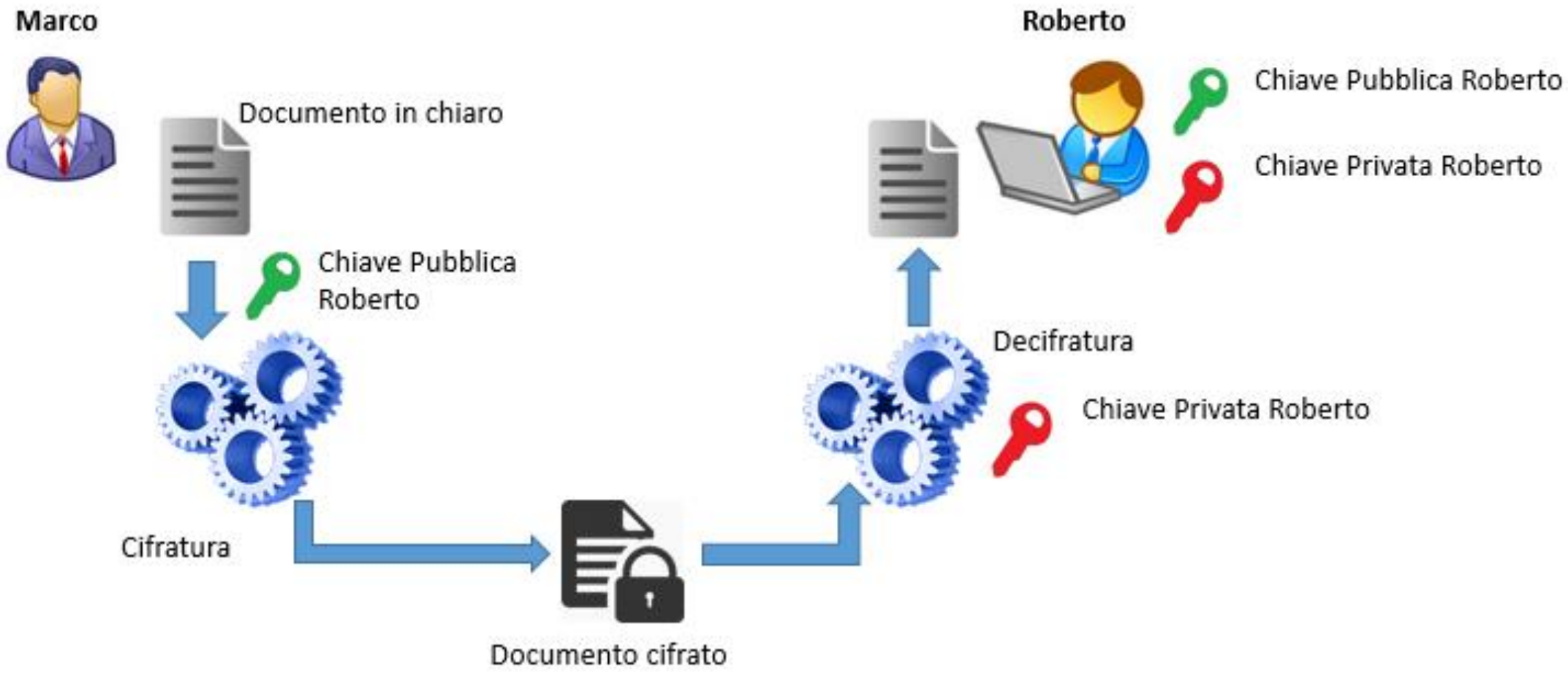
Bash - TERMINALE Client, Connessione al server

```
nc 127.0.0.1 1234
```

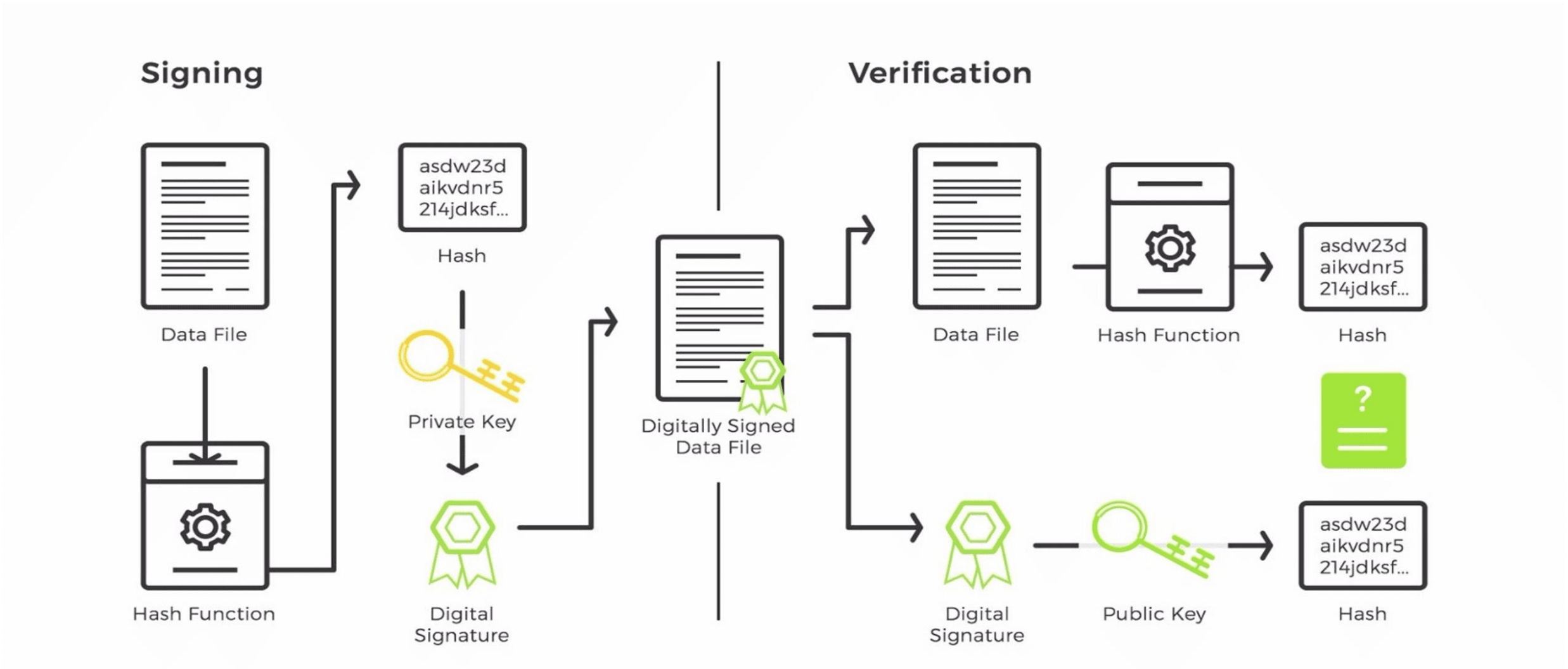
Crittografia a chiave simmetrica



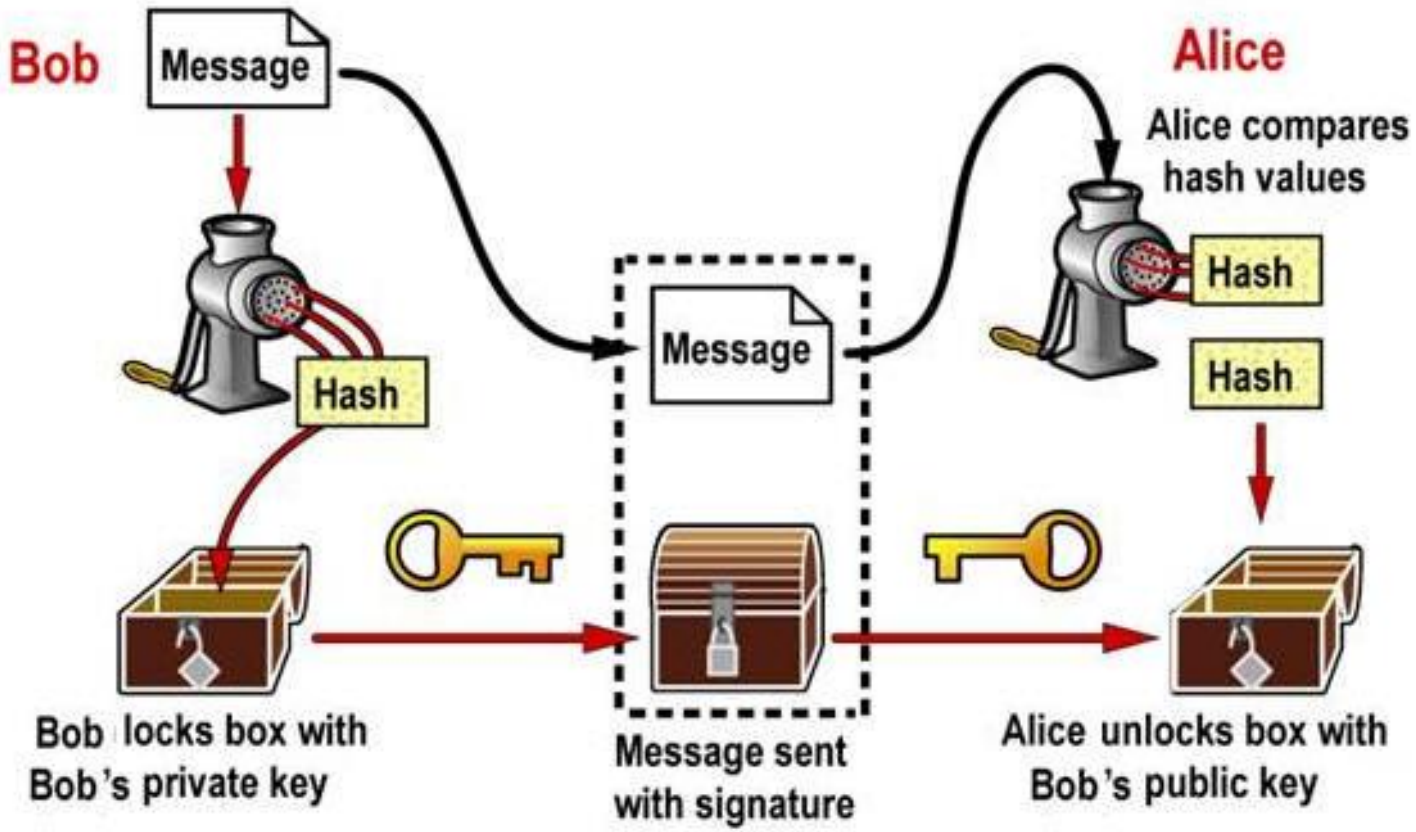
Crittografia a chiave asimmetrica (cifratura messaggi)



Crittografia a chiave asimmetrica (firma digitale)



Crittografia a chiave asimmetrica (firma digitale) II



I Digest

Una **digest hash function** (o semplicemente funzione di hash crittografica) è una funzione matematica che prende in ingresso un messaggio di lunghezza arbitraria e produce in uscita un valore di lunghezza fissa chiamato digest (o hash).

input (messaggio) → hash function → output (digest)

Il **digest** è una sorta di "impronta digitale" del messaggio: anche un piccolo cambiamento nel testo di partenza produce un risultato completamente diverso.

```
achz25@DESKTOP-L8GIS7I:~$ echo -n "ciao" | md5sum
6e6bc4e49dd477ebc98ef4046c067b5f  -
achz25@DESKTOP-L8GIS7I:~$ md5sum abc.txt
6714ac8929e1cb0a496064f44819bf0e  abc.txt
achz25@DESKTOP-L8GIS7I:~$
```

I Digest

Proprietà fondamentali di una funzione di hash crittografica

Per essere sicura e utile in ambito crittografico, una funzione di hash deve rispettare le seguenti proprietà:

- **Preimage resistance (resistenza alla preimmagine)**
  Dato un digest h, deve essere computazionalmente impossibile trovare un messaggio m tale che:
  hash(m) = h

- **Second preimage resistance (resistenza alla seconda preimmagine)**
  Dato un messaggio m_1, deve essere impossibile trovare un altro messaggio $m_2 \neq m_1$ tale che:
  $hash(m_1) = hash(m_2)$

- **Collision resistance (resistenza alle collisioni)**
  Deve essere estremamente difficile trovare due messaggi qualsiasi m_1 e m_2 tali che:
  $hash(m_1) = hash(m_2)$

Base64

Base64 è un sistema di codifica che consente la traduzione di dati binari (contenenti sequenze di 8 bit) in stringhe di testo ASCII, rappresentando i dati sulla base di sessantaquattro caratteri ASCII diversi.

La codifica Base64 consente la trasmissione di dati binari su un canale di comunicazione che supporta solo testo.

Applicazioni del Base64

- **Web e HTML/CSS:** usato per incorporare immagini o file binari direttamente nel codice.
- **Email:** consente l'invio di allegati tramite SMTP (solo 7-bit ASCII).
- **File di testo e XML:** inserisce dati binari (es. immagini, PDF) senza file esterni.
- **SVG:** può includere immagini raster codificate in Base64.
- **Data URI:** rappresenta contenuti binari in formato testuale.
- **Clipboard:** trasferimento rapido di piccoli dati binari (es. chiavi crittografiche).
- **Verifica umana:** rappresentazione leggibile di hash o impronte digitali.
- **QR Code:** codifica sicura e compatibile per dati binari.

Base64, algoritmo

L'algoritmo che effettua la conversione suddivide il file in gruppi da 6 bit, i quali possono quindi contenere valori da zero a sessantatré. Ogni possibile valore viene convertito in un carattere ASCII secondo la seguente tabella:

Valore	ASCII	Valore	ASCII	Valore	ASCII	Valore	ASCII
0	A	16	Q	32	g	48	w
1	B	17	R	33	h	49	x
2	C	18	S	34	i	50	y
3	D	19	T	35	j	51	z
4	E	20	U	36	k	52	0
5	F	21	V	37	l	53	1
6	G	22	W	38	m	54	2
7	H	23	X	39	n	55	3
8	I	24	Y	40	o	56	4
9	J	25	Z	41	p	57	5
10	K	26	a	42	q	58	6
11	L	27	b	43	r	59	7
12	M	28	c	44	s	60	8
13	N	29	d	45	t	61	9
14	O	30	e	46	u	62	+
15	P	31	f	47	v	63	/

Il set di sessantaquattro caratteri scelto per rappresentare i sessantaquattro valori (0-63) varia a seconda dell'implementazione scelta. L'obiettivo solitamente è di scegliere sessantaquattro caratteri che siano comuni alla maggior parte delle codifiche e che siano anche stampabili.

Base64, algoritmo

Esempio di codifica [modifica | modifica wikitesto]

Dati 3 byte che contengono i valori A, B, C, partendo dalla loro rappresentazione binaria, applichiamo l'algoritmo:

```
Rappresentazione ASCII:                  A           B           C
                                         |           |           |
Rappresentazione binaria:             01000001    01000010    01000011
                                        /    \      /    \      /    \
Suddivisione in gruppi da 6 bit:     010000  01  0100  0010  01  000011
                                     |----|  \------/  \------/  |----|
I 4 valori dedotti:                  010000   010100    001001   000011
                                       \/       \/        \/       \/
Il valore decimale:                    16       20         9        3
                                        |        |         |        |
Il valore codificato:                   Q        U         J        D
```

Quindi la sequenza di valori (`ABC`) viene convertita nella sequenza (`QUJD`).

L'algoritmo causa un aumento delle dimensioni dei dati del 33%, poiché ogni gruppo di 3 byte viene convertito in quattro caratteri. Questo supponendo che per rappresentare un carattere si utilizzi un intero byte.